Quantstamp DRAFT

Nillion - Merkle Claim

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Staking
Timeline	2026-03-11 through 2026-03-23
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• 01builders/nillion-merkledrop #b4fbf78
Auditors	• Andrei Stefan Auditing Engineer • Andy Lin Senior Auditing Engineer • Yamen Merhi Auditing Engineer

Documentation quality	Undetermined
Test quality	Undetermined
Total Findings	8 Unresolved: 8
High severity findings	0
Medium severity findings	1 Unresolved: 1
Low severity findings	2 Unresolved: 2
Undetermined severity findings	0
Informational findings	5 Unresolved: 5

Summary of Findings

The security audit has uncovered several considerations within the contracts, primarily concerning `TrustlessMerkleClaim` and its dependency `CosmosVerifier`. The contract implements a robust on-chain verification pipeline combining `ADR-036` signature verification, merkle proof validation, and cosmos address derivation for both single-key and multisig claim paths.

Recommendations largely focus on operational safeguards around deployment parameters. The immutability of `merkleRoot`, `bech32Prefix` and the operator role means any misconfiguration at deployment time is unrecoverable without redeploying an entirely new contract. Additionally, ensuring address canonicalization consistency between the Merkle tree generator and the on-chain verification, excluding unsupported account types from the snapshot, and adding a finalization mechanism before token recovery would substantially elevate the integrity of the claim process. Overall, addressing these considerations and implementing the recommendations will enhance security, ensuring greater protection for claimants and reducing operational risk.

ID	DESCRIPTION	SEVERITY	STATUS
NIL-1	Spurious Varint in Multisig Amino Encoding Breaks All Multisig Claims	Medium	Unresolved
NIL-2	Funder Role Front-Running Enables Fund Theft After Pause	Low	Unresolved
NIL-3	Signature Not Bound to Claim Index Enables Front-Runner Destination Override to Safe Previous Owner	Low	Unresolved
NIL-4	Missing `address(0)` Check on `ecrecover` Return Value	Informational	Unresolved
NIL-5	Non-Canonical Compressed Public Keys Accepted	Informational	Unresolved
NIL-6	Multisig Amino Varint Encoding Fails for More than 409 Keys	Informational	Unresolved

ID	DESCRIPTION	SEVERITY	STATUS
NIL-7	Case Sensitivity Inconsistency Between Address Verification and Merkle Proof	Informational	Unresolved
NIL-8	Duplicate Public Keys in Multisig Allow Single-Signer Threshold Bypass	Informational	Unresolved

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/01builders/nillion-merkledrop`

Included Paths: `contracts/TrustlessMerkleClaim.sol`

Operational Considerations

1. **ADR-036 Signature Verification**: TrustlessMerkleClaim uses on-chain Cosmos ADR-036 signature verification via CosmosVerifier library. The verification relies on three EVM precompiles: SHA256 (0×02), RIPEMD160 (0×03), and MODEXP (0×05). The claim message includes the contract address and chainId for replay protection. Low-s enforcement (EIP-2) is applied, which is compatible with standard Cosmos wallet implementations (Keplr, Leap, cosmjs).

2. **Double-Hash Merkle Leaf Construction**: TrustlessMerkleClaim uses `keccak256(bytes.concat(keccak256(abi.encode(index, cosmosAddressHash, amount))))` — the OpenZeppelin standard double-hash pattern that prevents second preimage attacks on internal Merkle tree nodes.
3. **EVM Precompile Availability**: CosmosVerifier depends on SHA256 (0×02), RIPEMD160 (0×03), and MODEXP (0×05) precompiles being available and functioning correctly on the deployment chain. These are standard EVM precompiles available on Ethereum mainnet and most L2s, but should be verified for non-standard chains.
4. **Multisig Claim Verification Pipeline**: `claimMultisig()` implements a 4-step verification: (1) derive multisig address from ordered compressed pubkeys + threshold via amino encoding (`sha256(amino_bytes)[:20]`), (2) verify Merkle proof, (3) verify threshold-many ADR-036 signatures from distinct signers (strictly ascending `signerIndices`), (4) mark claimed and transfer. Each signer signs with their individual cosmos address as the ADR-036 `signer` field, not the multisig address. Pubkey ordering matters — different ordering produces a different multisig address. Cross-path replay between `claim()` and `claimMultisig()` is not possible because single-signer addresses use `ripemd160(sha256(pubkey))` while multisig addresses use `sha256(amino_bytes)[:20]` .
5. **Immutable Operator in TrustlessMerkleClaim**: The `operator` role is set once in the constructor and cannot be transferred, rotated, or revoked. If the operator key is lost, `pause()` , `unpause()` , `recoverERC20()` , and `recoverUnclaimedTokens()` become permanently inaccessible. If compromised, the operator can pause the contract and drain all remaining NIL in a single transaction. The `operator` storage variable is not declared with the `immutable` keyword (though it is effectively immutable due to having no setter).
6. **Immutable Merkle Root and Bech32 Prefix With No Migration Path**: `merkleRoot` and `bech32Prefix` are set once in the TrustlessMerkleClaim constructor and cannot be updated. Any mismatch in address canonicalization, prefix selection, leaf encoding, or signed message format at deployment time permanently prevents affected users from passing verification. There is no corrective migration mechanism — the only recourse is redeploying a new contract with corrected parameters and funding it with a fresh token allocation.
7. **Token Recovery Available While Claims Are Active**: `recoverUnclaimedTokens()` requires only that the contract is paused, not that claims have been permanently finalized or that a deadline has passed. The operator can pause, drain the full NIL balance via `recoverUnclaimedTokens()` , then unpause — leaving the contract unable to honor remaining valid claims. The operator is trusted, but there is no on-chain mechanism enforcing that recovery only happens after all legitimate claims have been fulfilled.
8. **Address Canonicalization Dependency in Merkle Verification**: `_verifyPubkeyMatchesAddress()` compares Cosmos addresses case-insensitively, while `_verifyProof()` hashes the raw `cosmosAddress` bytes exactly as provided. The claim flow therefore depends on a single canonical textual representation being used consistently when generating the Merkle tree and when submitting claims. If semantically equivalent variants of the same address (e.g., mixed-case bech32) are included with different raw byte forms, one Cosmos account could map to multiple leaves or face verification failures.
9. **Unsupported Cosmos Account Types Cannot Claim**: The contract supports two claim paths: `claim()` for single-key secp256k1 accounts and `claimMultisig()` for flat threshold multisigs with secp256k1 inner keys. This covers the vast majority of accounts (including vesting, Ledger, and validator accounts). However, two account types cannot claim and holders must transfer tokens to a supported account before chain halt: (a) **x/group policy accounts** — the x/group module is enabled on Nillion, and group policy accounts have derived addresses (ADR-028) with no private key, making ADR-036 signatures impossible; if these accounts appear in the snapshot but are not excluded, they create permanently unclaimable Merkle leaves locking tokens in the contract; (b) **nested multisigs** (multisig-of-multisig) — `claimMultisig()` requires all inner keys to be 33-byte compressed secp256k1 pubkeys, but a nested multisig has an amino-encoded multisig as an inner key (much larger than 33 bytes), causing address derivation failure.

Key Actors And Their Capabilities

1. **Cosmos Claimants (TrustlessMerkleClaim)**: Users with valid Cosmos address ownership can claim NIL tokens by providing a compressed secp256k1 public key, an ADR-036 signature, and a Merkle proof. The 4-step verification pipeline: (1) derive Cosmos address from pubkey and match, (2) verify Merkle proof, (3) verify ADR-036 signature with destination+contract+chainId binding, (4) mark claimed and transfer. Claims are sent to an arbitrary EVM `destination` address specified and signed by the claimant. Cosmos multisig accounts can claim via `claimMultisig()` , which verifies threshold-many ADR-036 signatures from distinct signers against amino-encoded multisig address derivation.
2. **TrustlessMerkleClaim Operator**: Set immutably in the constructor. Can pause/unpause the contract, recover non-NIL ERC20 tokens (`recoverERC20`), and recover all remaining NIL tokens when paused (`recoverUnclaimedTokens` — sends full balance to operator). The contract no longer uses `Ownable` ; the operator role replaces both the previous owner and funder roles. There is no mechanism to transfer or rotate the operator.

Findings

NIL-1

Spurious Varint in Multisig Amino Encoding Breaks All Multisig Claims

Medium Unresolved

File(s) affected: `contracts/TrustlessMerkleClaim.sol`

Description: In `CosmosVerifier.deriveMultisigAddressBytes()` , the amino encoding inserts a `varint(innerLength)` between the 4-byte amino prefix (`22C1F7E2`) and the protobuf fields (lines 107-113). However, the canonical Cosmos amino encoding produced by Go's `amino.MarshalBinaryBare` does **not** include this length prefix. The correct encoding is simply: `aminoPrefix (4 bytes) || protobuf fields` .

As a result, `deriveMultisigAddressBytes()` produces an address that never matches any real on-chain Cosmos multisig address. When `TrustlessMerkleClaim.claimMultisig()` calls `_verifyMultisigAddress()` , the derived address will always differ from the claimed `cosmosAddress` , causing every multisig claim to revert with `PubkeyAddressMismatch` .

Exploit Scenario:
A user whose tokens are allocated to a Cosmos multisig address (e.g., a DAO treasury or multi-party escrow) attempts to call `claimMultisig()`. Despite providing valid pubkeys, threshold, signatures, and merkle proof, the call always reverts because the on-chain amino encoding produces a different address hash than the real Cosmos multisig address. All funds allocated to multisig addresses are permanently unclaimable.

Recommendation: Remove the `varint(innerLength)` from the amino encoding buffer. The correct layout is: `aminoPrefix (4 bytes) || 0x08 || varint(threshold) || [0x12 0x26 pubkeyAminoPrefix key]...` — with no length prefix between the amino type prefix and the protobuf fields.

NIL-2 Funder Role Front-Running Enables Fund Theft After Pause

• Low ⓘ Unresolved

File(s) affected: `contracts/TrustlessMerkleClaim.sol`

Description: In `TrustlessMerkleClaim.fund()` (line 318-324), the first caller to invoke `fund()` permanently becomes the `funder`. Since contract deployment and funding occur in separate transactions, an attacker can front-run the legitimate deployer's `fund()` call with `fund(1)` (sending just 1 wei of NIL) to capture the `funder` role.

If the deployer works around this hijack by sending NIL directly via `token.transfer(contract, amount)`, the claims still function correctly. However, when the owner eventually pauses the contract, the attacker, who holds the `funder` role, can call `TrustlessMerkleClaim.recoverUnclaimedTokens()` (line 346-350) to steal all remaining unclaimed NIL tokens. The attacker's cost is 1 wei of NIL; the potential theft is the entire remaining allocation.

Exploit Scenario:
1. Deployer deploys `TrustlessMerkleClaim` and submits a `fund()` transaction.
2. Attacker front-runs with `fund(1)`, becoming the permanent `funder`.
3. Deployer's `fund()` reverts with `AlreadyFunded`. Deployer sends NIL via direct `token.transfer()` as a workaround.
4. Claims proceed normally. After the claim period ends, the owner calls `pause()`.
5. Attacker calls `recoverUnclaimedTokens(attackerAddress, contractBalance)` and drains all remaining NIL.

Recommendation: Set the `funder` in the constructor or add an `onlyOwner` function `setFunder(address)` to allow the owner to override the funder. Alternatively, deploy and fund atomically via a factory contract.

NIL-3 Signature Not Bound to Claim Index Enables Front-Runner Destination Override to Safe Previous Owner

• Low ⓘ Unresolved

File(s) affected: `contracts/TrustlessMerkleClaim.sol`

Description: `TrustlessMerkleClaim.claim()` verifies a Cosmos ADR-036 signature over a message constructed in `_buildClaimMessage()` that includes the `cosmosAddress`, destination, contract address, and chain ID. However, the signed message does not include the merkle index or amount. The merkle leaf binds (index, cosmosAddressHash, amount) independently from the signature, which only binds (cosmosAddress, destination).

If the same Cosmos address appears in the merkle tree under multiple indices, a single valid signature authorizes claiming any of those indices to the signed destination. Once a user broadcasts a claim transaction for index 1 with destination 0xAA, the signature becomes visible on-chain (or in the mempool). A malicious actor can extract that signature and submit a claim for a different index (e.g., index 50) belonging to the same Cosmos address, forcing those tokens to `0xAA`. All verification steps pass because the reconstructed message is identical, it contains no index or amount to differentiate the two claims.

If the user wants to re-route funds to a different address (`0xBB`) for any reason such as sending to different cold wallet, or a third party, a malicious attacker can stop them from doing it. Anyone who observes the first claim's signature can front-run and claim the remaining indices to the original destination, removing the user's ability to choose where those tokens go.

Recommendation:
- Include index and amount in the signed message so that each signature authorizes exactly one specific claim entry and cannot be reused across indices.
- Alternatively, if the merkle tree is guaranteed to contain at most one entry per Cosmos address, document this constraint as a security assumption.

NIL-4 Missing `address(0)` Check on `ecrecover` Return Value

• Informational ⓘ Unresolved

File(s) affected: `contracts/TrustlessMerkleClaim.sol`

Description: `CosmosVerifier.verifySignature()` (line 229-263) calls `ecrecover` and compares the result to `expectedAddress` without first checking whether `ecrecover` returned `address(0)`. If `ecrecover` fails (e.g., due to an invalid signature), it returns `address(0)`. For this to be exploitable, an attacker would need to find a valid compressed public key whose `keccak256(x || y)` truncates to `address(0)` — a computation requiring ~2^128 operations, making it infeasible in practice.

NIL-5 Non-Canonical Compressed Public Keys Accepted

• Informational ⓘ Unresolved

Description: `CosmosVerifier.decompressPubkey()` (line 184) does not validate that the x-coordinate of a compressed public key is less than the secp256k1 field prime `P`. A non-canonical pubkey where `x >= P` would produce a different Cosmos address than any legitimate key. However, exploitation is prevented by the merkle proof: an attacker would need a valid merkle leaf for the non-canonical address, which requires control of the merkle tree (a trusted deployer action).

Recommendation: Add a check: `require(x < P, "non-canonical pubkey")` at the start of `decompressPubkey()`.

NIL-6
Multisig Amino Varint Encoding Fails for More than 409 Keys

• Informational ⓘ Unresolved

Description: In `CosmosVerifier.deriveMultisigAddressBytes()` (lines 94, 108-113), the varint encoding for `innerLength` only handles 1-byte and 2-byte varints. When `numKeys >= 410`, the `innerLength` value exceeds the 2-byte varint maximum (16383), producing an invalid protobuf encoding. This causes the derived address to be incorrect and the claim to revert. In practice, no legitimate Cosmos multisig has more than 409 keys.

Recommendation: Add an explicit upper bound: `require(numKeys <= 409)`, or implement full multi-byte varint encoding.

NIL-7
Case Sensitivity Inconsistency Between Address Verification and Merkle Proof

• Informational ⓘ Unresolved

Description: In `TrustlessMerkleClaim._verifyPubkeyMatchesAddress()` (line 202) and `_verifyMultisigAddress()` (line 148), the pubkey-to-address comparison uses `_toLowerCase()` for case-insensitive matching. However, `TrustlessMerkleClaim._verifyProof()` (line 219) hashes `cosmosAddress` case-sensitively via `keccak256(bytes(cosmosAddress))`. If a deployer includes the same address with different casings as separate merkle entries, this inconsistency could enable double-claiming. Since the deployer is already trusted with the entire merkle tree construction, this is a footgun rather than an external vulnerability.

Recommendation: Normalize `cosmosAddress` to lowercase before hashing for merkle leaf construction in `_verifyProof()`.

NIL-8
Duplicate Public Keys in Multisig Allow Single-Signer Threshold Bypass

• Informational ⓘ Unresolved

Description: In `TrustlessMerkleClaim._verifyMultisigSignatures()` (line 167), the sorted-index check at line 184 (`signerIdx <= prevIndex`) only prevents reusing the same *array position* twice (e.g., `signerIndices = [2, 2]` is rejected). It does not prevent the same *public key* from appearing at multiple positions in `compressedPubkeys[]`. If a caller supplies `compressedPubkeys = [keyA, keyA, keyA]` with `threshold = 2` and `signerIndices = [0, 1]`, both signatures verify against `keyA`, satisfying the threshold with a single key holder.

This is not exploitable in practice because `TrustlessMerkleClaim._verifyMultisigAddress()` (line 148) derives the multisig address from the supplied pubkeys via `CosmosVerifier.deriveMultisigAddressBytes()`, and the resulting degenerate address must match a leaf in the merkle tree. The Cosmos SDK prevents creating multisigs with duplicate keys, so no legitimate on-chain snapshot would contain such an address.

Recommendation: Add a uniqueness check on `compressedPubkeys[]` entries at the start of `claimMultisig()` — for example, require keys to be in sorted lexicographic order (which matches Cosmos SDK's canonical key ordering) and reject adjacent duplicates.

Auditor Suggestions

S1 `recoverERC20()` Callable While Contract Is Unpaused

Unresolved

File(s) affected: `contracts/TrustlessMerkleClaim.sol`

Description: `TrustlessMerkleClaim.recoverERC20()` (line 338) has the `onlyOwner` modifier but no `whenPaused` modifier, unlike `TrustlessMerkleClaim.recoverUnclaimedTokens()` (line 346) which requires the contract to be paused. The function explicitly blocks recovery of the NIL claim token (`require(erc20 != address(token))`), so it cannot drain claim funds. This asymmetry appears intentional — the owner can rescue mistakenly-sent non-NIL tokens without disrupting active claims — but the design intent is not documented.

Recommendation: Document that the asymmetry is intentional. If it is not, add `whenPaused` to `recoverERC20()` for consistency with `recoverUnclaimedTokens()`.

S2 Immutable Operator with No Transfer Mechanism in `Trustlessmerkleclaim` Unresolved

Description: In the latest TrustlessMerkleClaim (commit 3e1c257), the `operator` address is set in the constructor and stored as a regular `public` storage variable with no setter function. The `operator` controls all privileged operations: `pause()`, `unpause()`, `recoverERC20()`, and `recoverUnclaimedTokens()`. There is no mechanism to transfer, rotate, or revoke the operator role after deployment.

If the operator private key is lost, all admin functions become permanently inaccessible — the contract cannot be paused in emergencies, and unclaimed/accidentally-sent tokens are locked forever. If the key is compromised, the attacker can call `pause()` then `recoverUnclaimedTokens()` in a single transaction to drain the entire NIL balance, since `recoverUnclaimedTokens()` sends the full `token.balanceOf(address(this))` to the operator address.

The previous version used OpenZeppelin's `Ownable` which included `transferOwnership()`. The refactor to a custom `operator` role removed this capability. Additionally, `operator` is not declared with the Solidity `immutable` keyword despite being effectively immutable because no setter exists.

Exploit Scenario:
Use OpenZeppelin's `Ownable2Step` for the operator role.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/01builders/nillion-merkledrop`

- `872...c4a ./contracts/TrustlessMerkleClaim.sol`

Changelog

- 2026-03-23 - Initial report
- 2026-03-27 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PandcakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 – Quantstamp, Inc.